

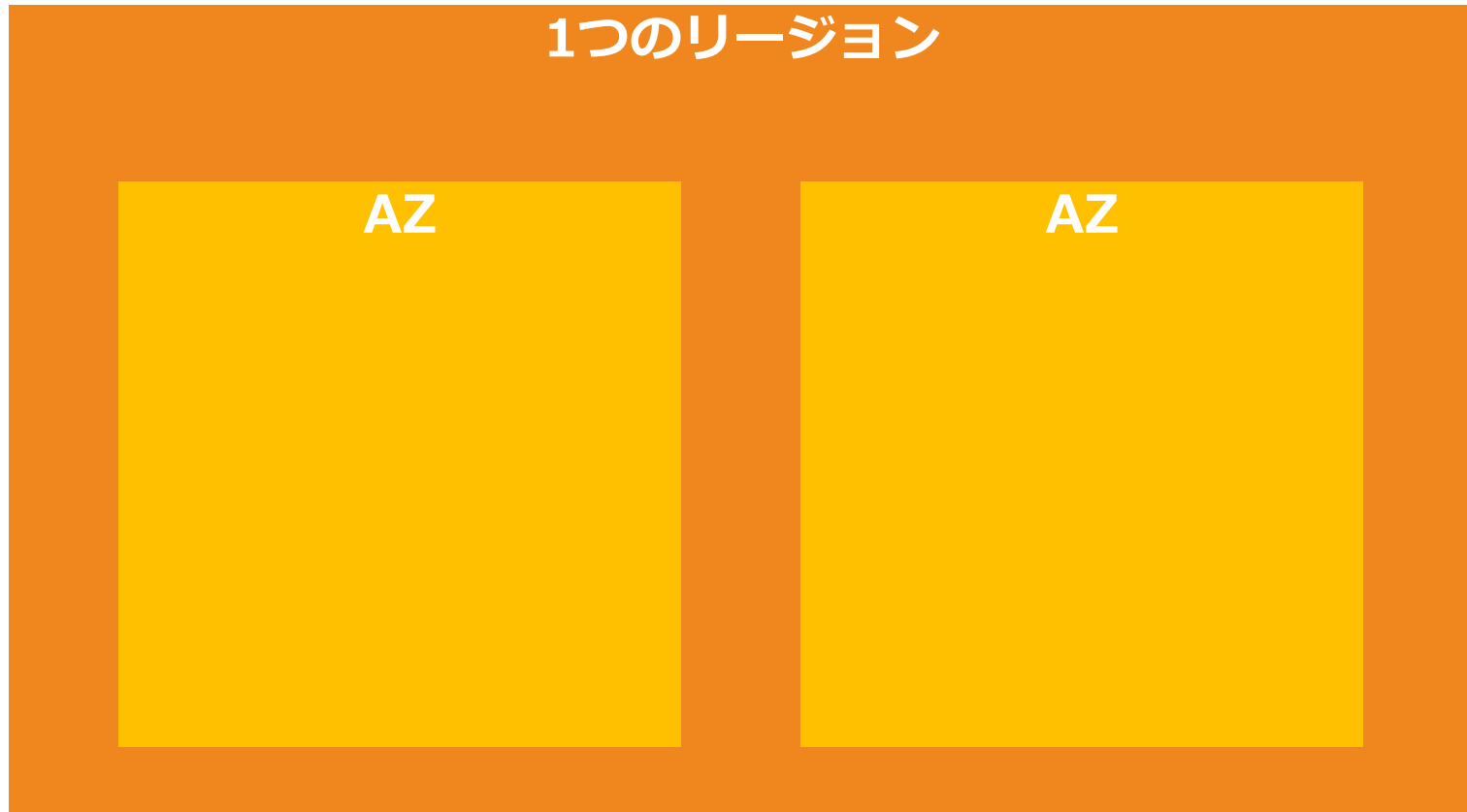
Well-Architected Framework

レクチャー	レクチャーで学ぶ内容
AWSアーキテクチャ設計の基礎	リージョン・AZ・VPCを利用したAWSの基本的なアーキテクチャ設計を確認します。
Well-Architected Framework	AWSが推奨するアーキテクチャ設計ガイドラインであるWell-Architected Frameworkを確認します。
AWS設計ケーススタディ	AWSのアーキテクチャ設計に関するケーススタディにより実践的な理解を深めます。

AWS アーキテクチャ設計 の基礎

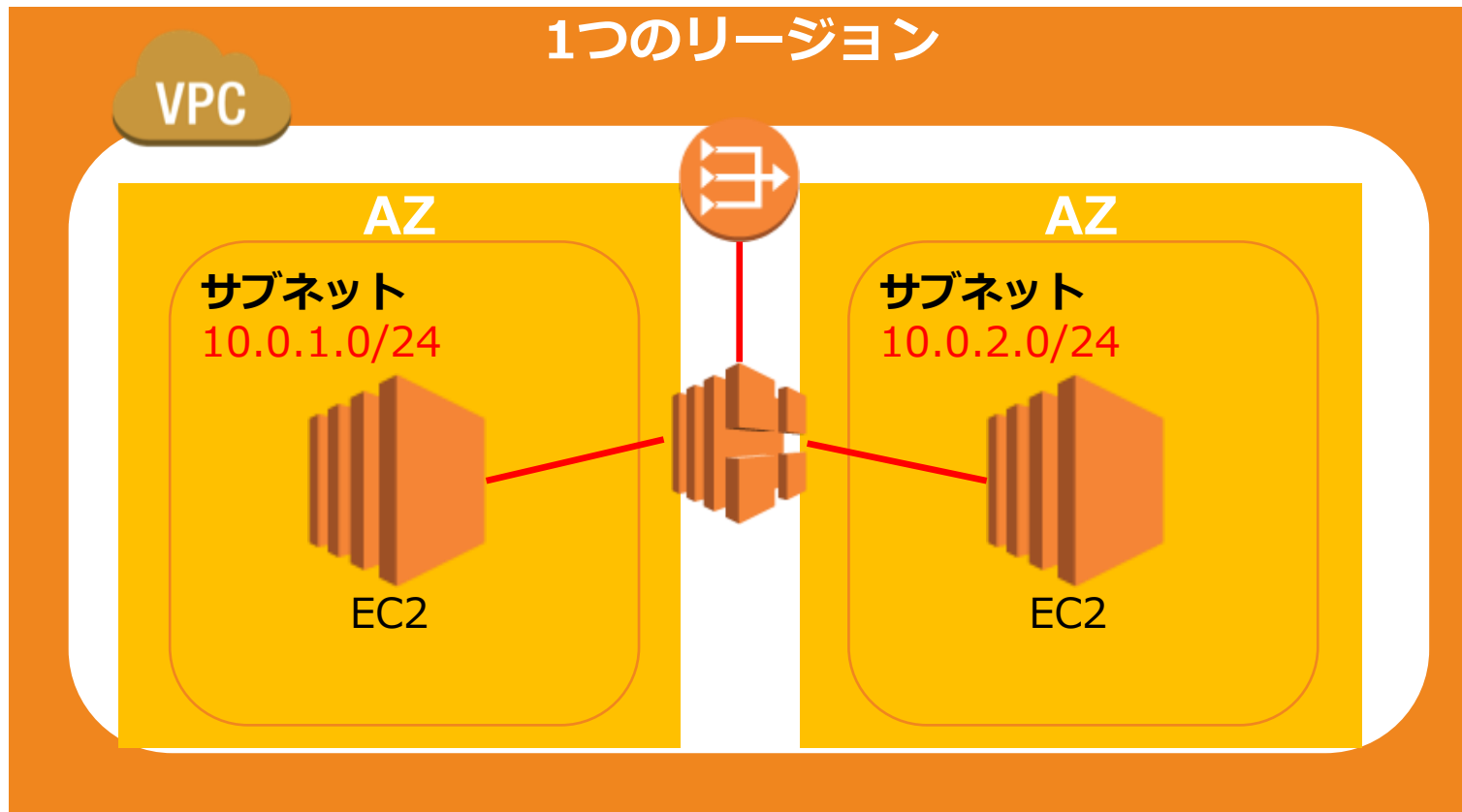
AZの選択

1つのリージョンにつき2つのAZを利用してアーキテクチャを設計することが基本（3つ以上はコスト効率が低下する）



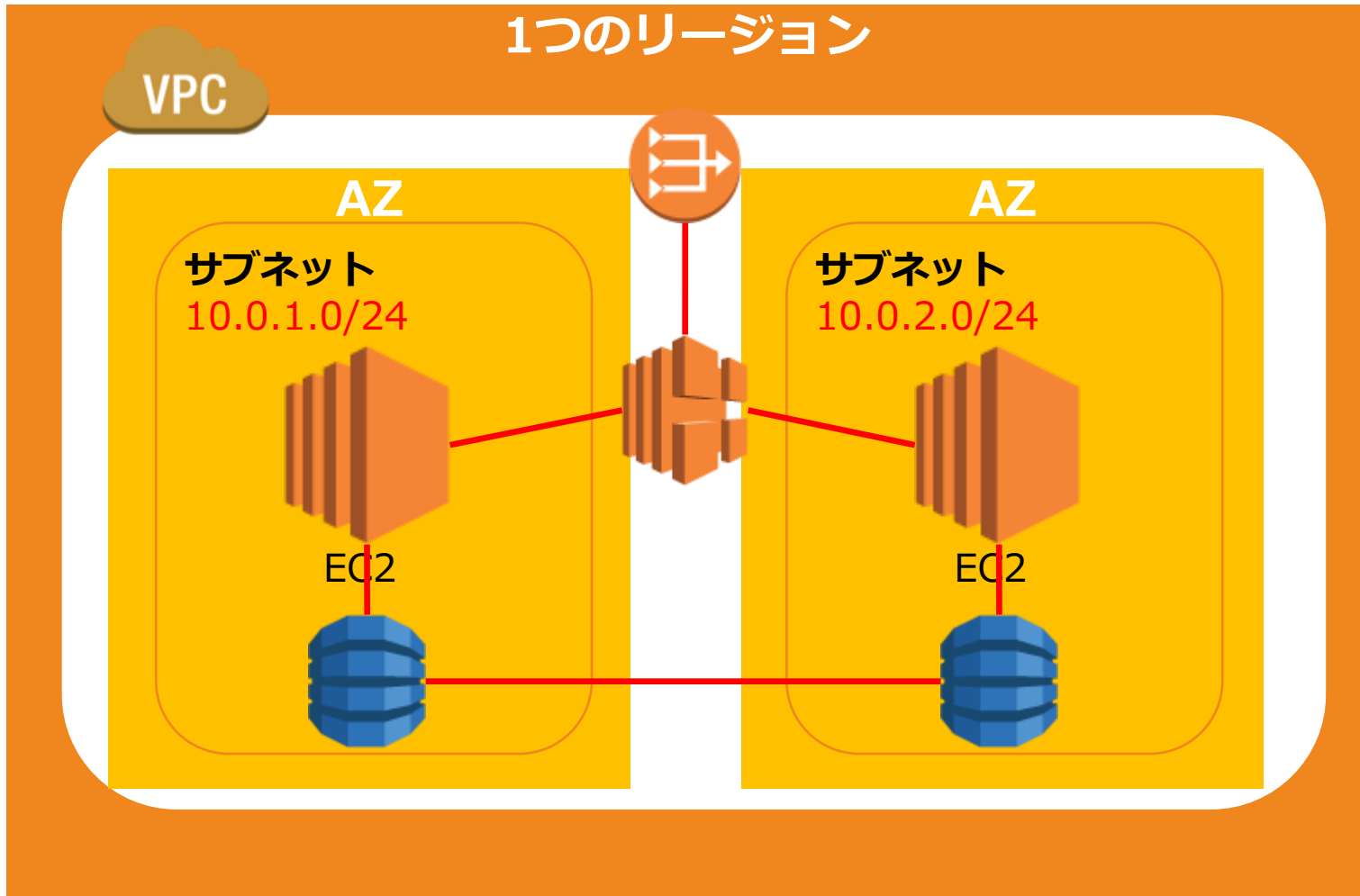
AZの選択

1つのリージョンにつき2つのAZを利用してアーキテクチャを設計することが基本（3つ以上はコスト効率が低下する）



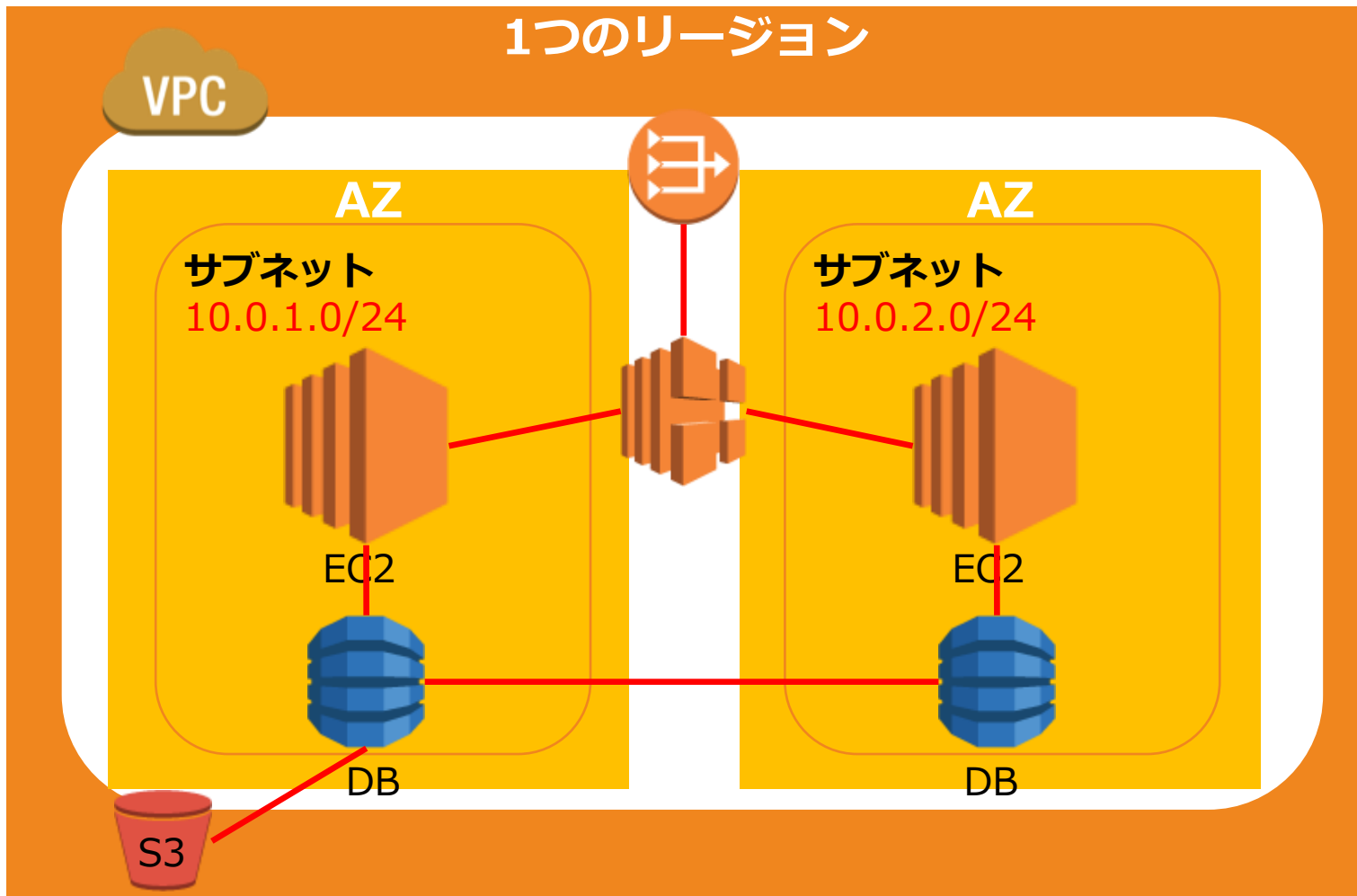
AZの選択

マルチAZにサーバーやDBの冗長構成を確立させることで高い可用性を実現する



AZの選択

マルチAZにサーバーやDBの冗長構成を確立させることで高い可用性を実現する



VPC

1つのVPCでアーキテクチャを設計するのが基本となる

1つのVPC

- 基本的には1つのVPCに1つのシステムやアプリケーションを配置する
- 一人などの小規模で利用する場合は2つ以上VPCを利用するのが面倒なケースもある

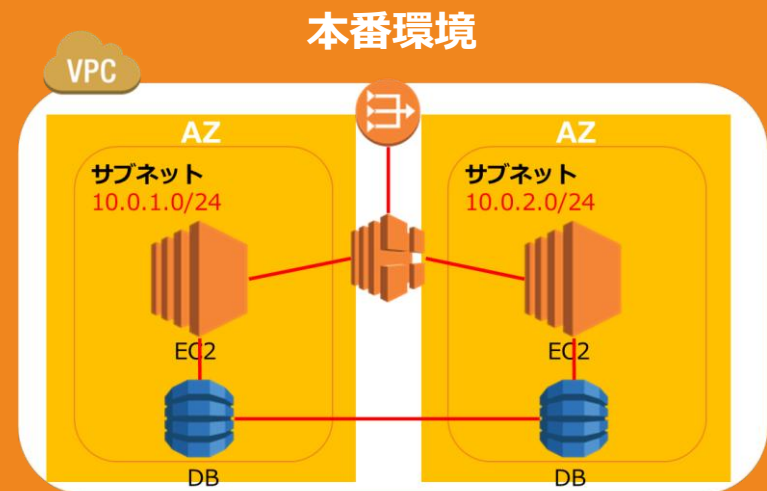
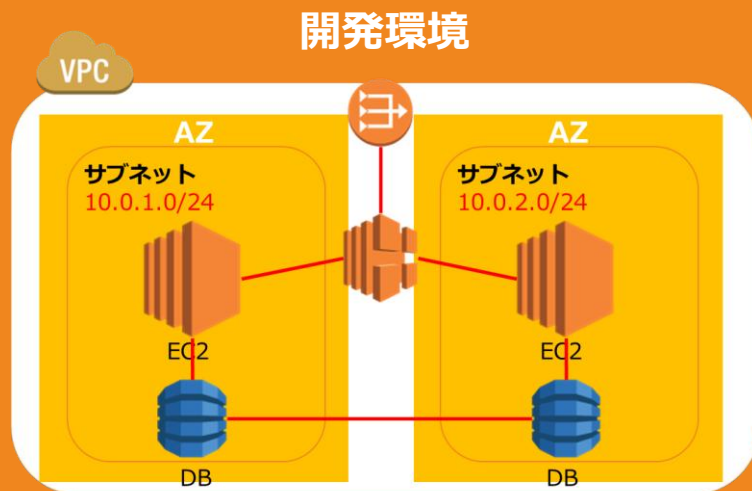
2つ以上のVPC

- システムを分けることが必要な場合や、テスト用や本番用などに分ける場合に利用する。
- 2つ以上のVPCで可用性を確保するのが適切なAWSアーキテクチャ設計となる

VPC

構成を分けた方がよいケースでは、2つ以上のVPCに分けてアーキテクチャを配置することで利用範囲を分割する

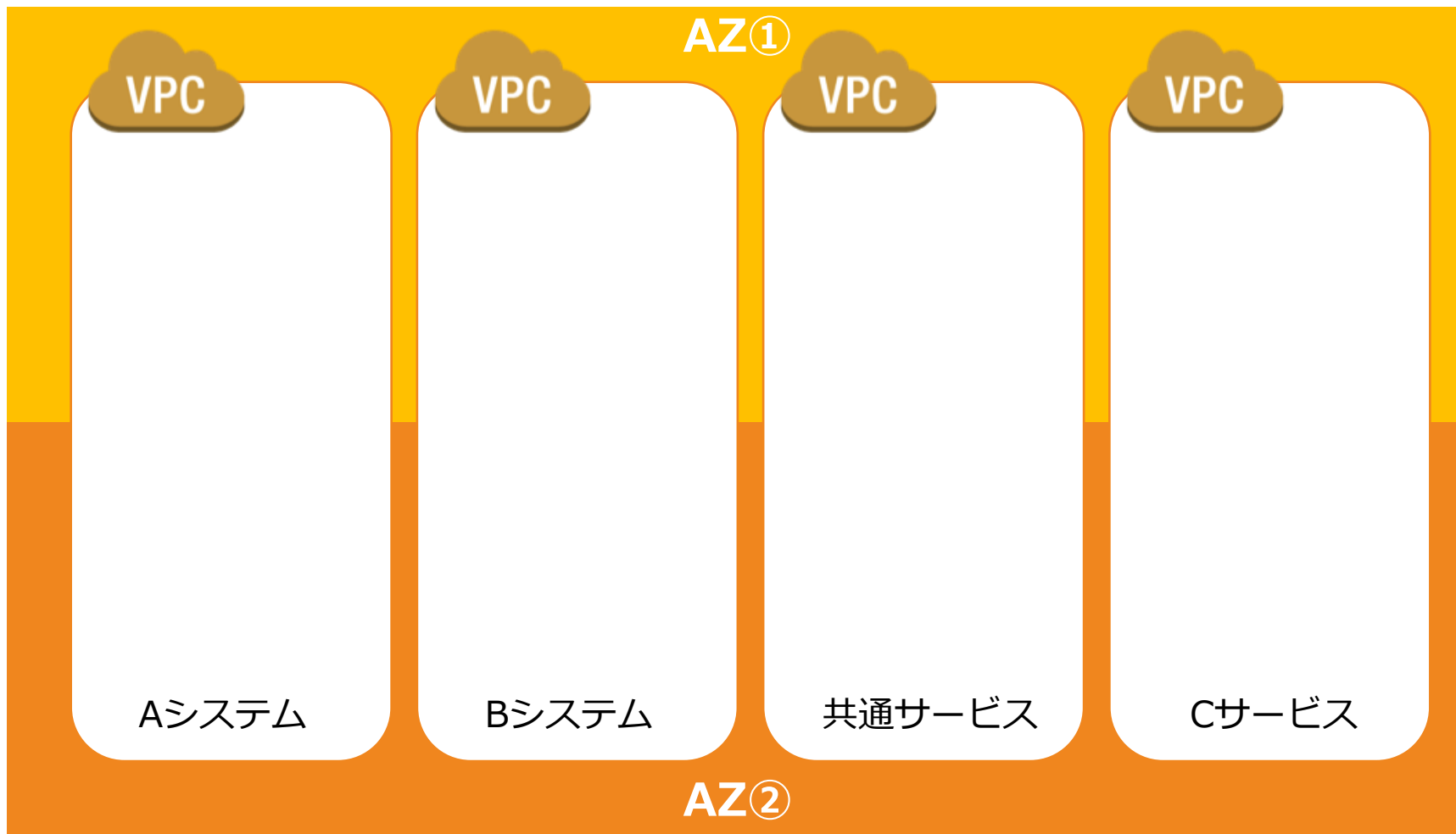
1つのリージョン



VPC

ネットワークの分割方法はマルチVPC方式とマルチアカウント方式を選択する

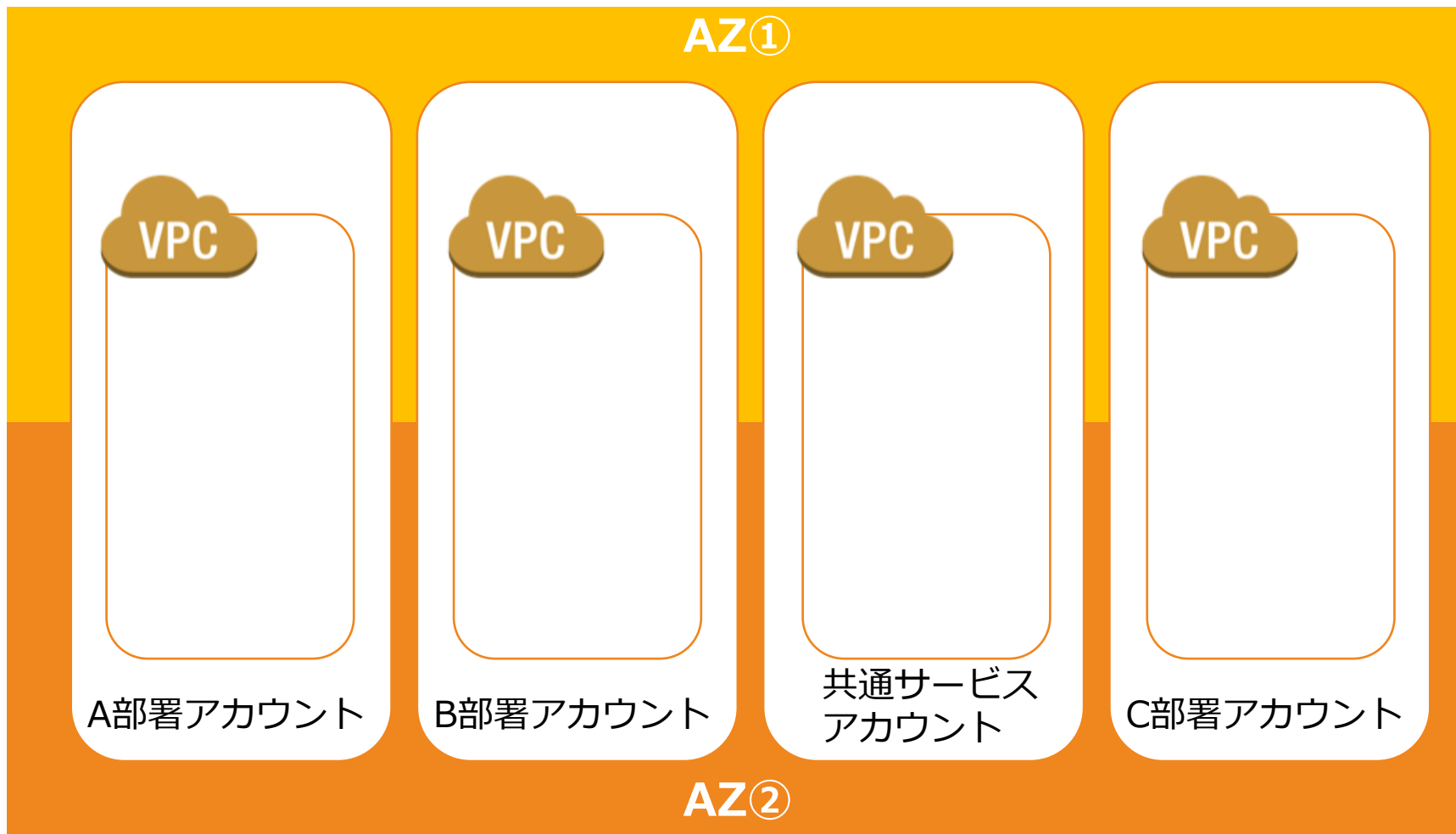
マルチVPC方式



VPC

ネットワークの分割方法はマルチVPC方式とマルチアカウント方式を選択する

マルチアカウント方式



サブネットの分割

サブネットはCIDR範囲で分割したネットワークセグメント

パブリックサブネット
10.0.1.0/24



EC2

トラフィックがインターネット
ゲートウェイにルーティングさ
れるサブネット

プライベートサブネット
10.0.2.0/24



EC2

インターネットゲートウェイへ
のルートがないサブネット

サブネットの分割

サブネットはインターネットアクセス範囲を定義するために利用する

パブリックサブネット
10.0.1.0/24



EC2

- インターネットと接続が必要なリソースを揃える
- インターネットとのアクセス制御に利用する

プライベートサブネット
10.0.2.0/24



EC2

- インターネットから隔離することでセキュリティを高める

サブネットの分割

WEBアクセスが必要か否かとセキュリティ対応に応じて、パブリックとプライベートで分割

パブリックサブネット
10.0.1.0/24



EC2

- ウェブアプリケーションのインターネットアクセス制御

プライベートサブネット
10.0.2.0/24



EC2

- データベース処理
- バッチ処理インスタンス
- バックエンドのインスタンス

サブネットの分割

サブネットのサイズは「/24」以上の大きいサブネットを推奨

パブリックサブネット
10.0.1.0/24



EC2

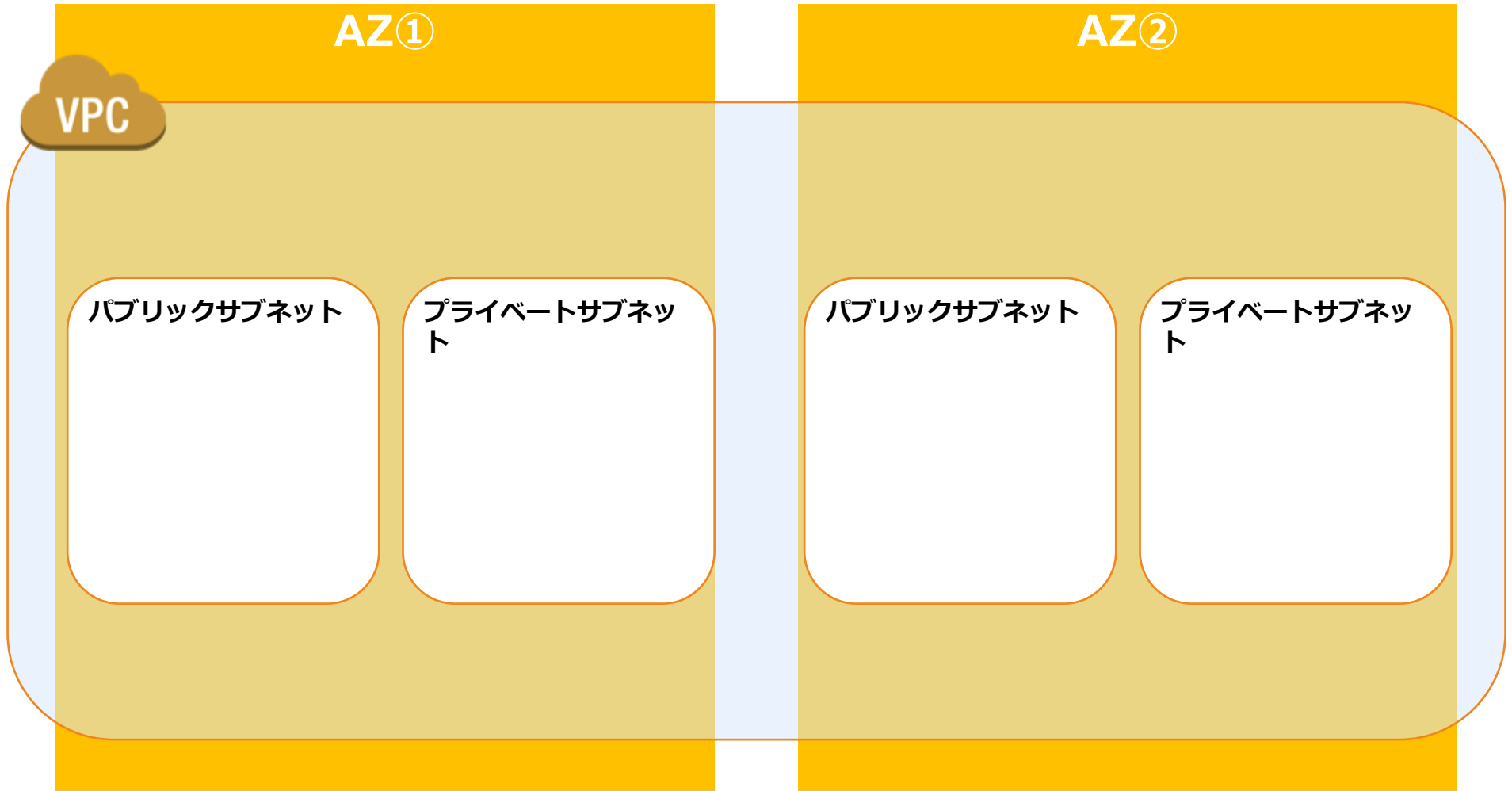
プライベートサブネット
10.0.2.0/24



EC2

サブネットの分割

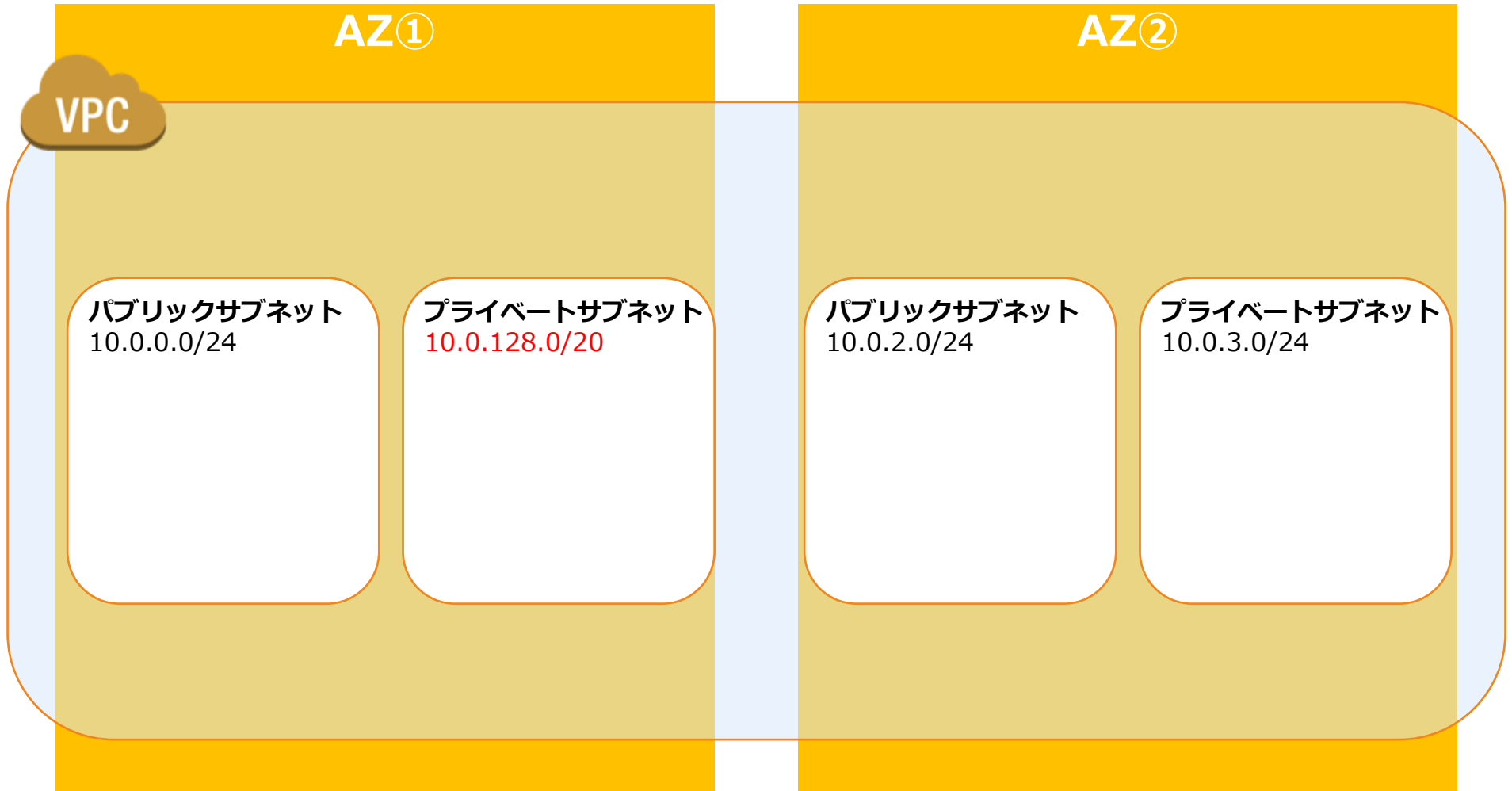
1つのAZに対して1つのパブリックサブネットと1つのプライベートサブネットが基本



サブネットの分割

プライベートサブネットに多くのIPアドレスを割り当てる

10.0.0.0/21



Well-Architected Framework

6つの設計原則の利用

アソシエイト試験はWell-Architected Frameworkで提唱されている6つの設計原則に沿った試験範囲になっている

Reliability
信頼性

Performance
Efficiency
パフォーマンス
効率

Security
安全性

Cost
Optimization
コスト最適化

Operational
Excellence
運用上の
優秀性

Sustainability
持続可能性

試験範囲

Well Architected Frameworkの6つの設計原則のうちの4つが試験範囲となっている。

分野		比率
分野 1	レジリエントアーキテクチャの設計	30%
分野 2	高パフォーマンスアーキテクチャの設計	28%
分野 3	セキュアなアプリケーションとアーキテクチャの設計	24%
分野 4	コスト最適化アーキテクチャの設計	18%

5つの設計原則

Well Architected Frameworkの6つの設計原則のうちの4つが試験範囲となっている。

設計原則

レジリエントアーキテクチャの設計

➡ Reliability

高パフォーマンスアーキテクチャの設計

➡ Performance Efficiency

セキュアなアプリケーションとアーキテクチャの設計

➡ Security

コスト最適化アーキテクチャの設計

➡ Cost Optimization

01版のレクチャー範囲

➡ Operational Excellence

なし

➡ Sustainability

Well-Architected Framework

Well-Architected Frameworkは次の3つの内容で構成される

①

Well-Architected
Framework
ホワイトペーパー



②

AWSのSAまたは
認定パートナーによる
支援制度

③

セルフチェック向けの
Well-Architected
Tool

Well-Architected Framework

ベストプラクティスを利用することで最適解や改善点を把握する。あくまで参考であり絶対というものではない。

要件定義	設計	構築	運用
✓AWSを利用したシステム要件を検討する材料としてホワイトペーパーを確認して最適な要件検討に利用する	✓設計においてホワイトペーパーを確認しながら、適切な設計方式を検討する	✓リリース前にAWSのインフラ構成にリスクや改善点がないかホワイトペーパーに基づいて確認する	✓運用中に定期的にホワイトペーパーに沿ったレビューを実施し、リスクや改善点がないか確認する

Reliability : 信頼性

障害による中断・停止と障害復旧による影響を軽減するインフラストラクチャーを構成する

【設計事項】

- ✓ インフラストラクチャサービスの障害復旧の自動化など軽減設計
- ✓ 復旧手順のテストによる検証
- ✓ 需要変化に応じた水平方向へのスケーラビリティによる高可用性の確保
- ✓ キャパシティの推測をやめる
- ✓ モニタリングと自動化を進める

Reliability : 信頼性

信頼性の基本対応領域は基盤、変更管理、障害管理の3つ

基盤

変更管理

障害管理

信頼性の主要サービス

基盤	IAM VPC Auto Scaling ELB Cloud Formation
変更管理	CloudTrail AWS Config
障害管理	Cloud Watch

Performance Efficiency : パフォーマンス効率

システム要件のリソース最適化によるインフラの効率化

【設計事項】

- ✓ システム要件を満たすためのコンピューティングリソースを効率化する
- ✓ システム要件やAWSサービスの進化に応じてAWSインフラの効率化を推進する
 - 先端技術の一般化
 - グローバル化を即座に達成
 - サーバレスアーキテクチャの利用
 - より頻繁な実験

Performance Efficiency : パフォーマンス効率

パフォーマンス効率の基本対応領域はコンピューティング、ストレージ、データベース、容量と時間のトレードオフの4つ

コンピューティング

ストレージ

データベース

容量と時間の
トレードオフ

パフォーマンス効率の主要サービス

コンピューティング	Auto Scaling	Lambda			
ストレージ	EBS	S3	Glacier	EFS	
データベース	RDS	Dynamo DB	Elastic search	Aurora	Redshift
容量と時間のトレードオフ	Cloud Front	Elasti Cache			

Security : セキュリティ

AWS内のデータ／システム／アセットの保護とモニタリングによりセキュリティを高める

【設計事項】

- ✓ 全てのレイヤーにおいてセキュリティを適用
- ✓ アクセス追跡・モニタリングの確実な実施
- ✓ 条件ドリブンのアラートをトリガーとしてセキュリティイベントへの応答を自動化
- ✓ AWS責任共有モデルに基づく対象範囲の保護に集中する
- ✓ セキュリティのベストプラクティスの自動化
 - ソフトウェアベースのセキュリティ設定を使用し、迅速でコスト効率の良いスケーリングを安全に実行する
 - 仮想サーバーのカスタムベースラインイメージによる新サーバーへの適用自動化
 - インフラストラクチャ全体のテンプレ化による管理

Security : セキュリティ

セキュリティの基本対応領域はデータ保護、権限管理、インフラ保護、検出制御の4つ

データ保護

権限管理

インフラ保護

検出制御

セキュリティの主要サービス

データ保護	ELB	EBS	S3	RDS	KMS
権限管理	IAM	MFA			
インフラ保護	VPC				
検出制御	Cloud Trail	Cloud Watch	AWS GuardDuty	Amazon Inspector	

Cost Optimization : コスト最適化

不必要なリソースの削減や最適な料金選択によりコストを削減

【設計事項】

- ✓ 不必要なリソース削減
- ✓ 透明性のある費用賦課
- ✓ マネージド型サービスの利用によるコスト削減
- ✓ 固定の償却コストを変動コストへと転換
- ✓ スケールによるコストメリット
- ✓ データセンターへの投資不要化

Cost Optimization : コスト最適化

コスト最適化の基本対応領域は需要と供給の一致、コスト効率の高いリソース、支出の認識、継続した最適化の4つ

需要と供給の一致

コスト効率の
高いリソース

支出の認識

継続した最適化

コスト最適化の主要サービス

需要と供給 の一致	Auto Scaling
コスト効率 の高いリソース	EC2 購入方式 Trusted Advisor
支出の認識	Cloud Watch 見積もり ツール
継続した 最適化	AWS 最新情報 Trusted Advisor

Operational Excellence：運用上の優秀性

運用上の優秀性とは計画変更が起こった場合や予期せぬイベントの発生時において、自動化された運用実務及び文書化されテストされレビューされた手順があること

【設計事項】

- ✓ コードに基づく運用実施
- ✓ ビジネス目的に沿った運用手順
- ✓ 定期的かつ小規模で増加的な変更実施
- ✓ 予期せぬイベントへの応答テスト
- ✓ 運用イベントと障害からの学習
- ✓ 運用手順を最新のものに保持すること

Operational Excellence：運用上の優秀性

運用上の優秀性の基本対応領域は準備、運用、進化の3つ

準備

運用

進化

運用上の優秀性の主要サービス

準備	Cloud Formation Code シリーズ Runbook Playbook
運用	System Manager Service Catalog CloudTrail AWS Artifact AWS GuardDuty Cloud Watch AWS Config API Gateway
進化	継続的かつ段階的な改善のために時間とリソースを割り当て、運用の有効性と効率性を向上させる

6つの設計原則

AWS活用では6つの設計原則を常に念頭に置くことが重要

Reliability
信頼性

Performance
Efficiency
パフォーマンス
効率

Security
安全性

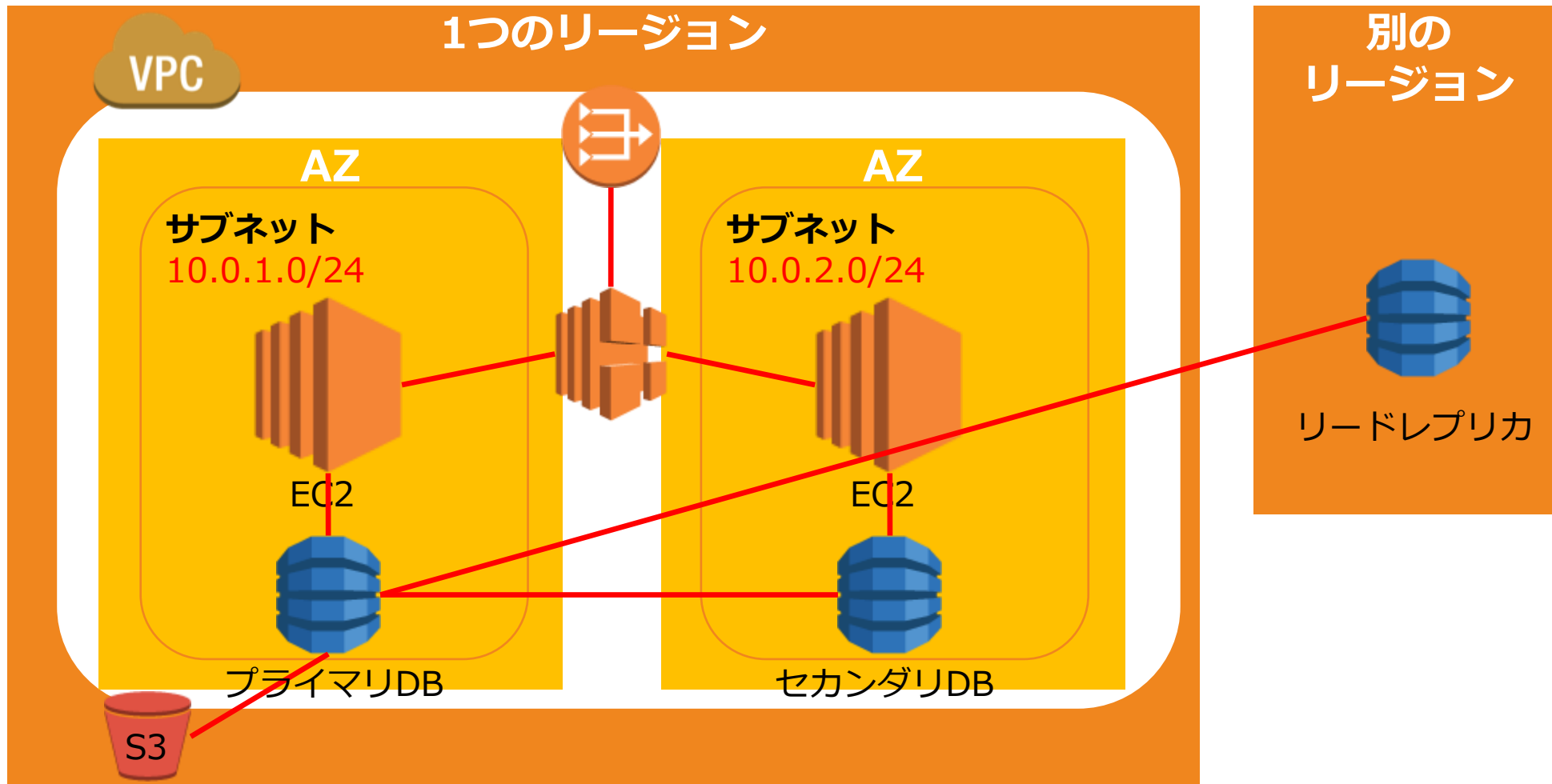
Cost
Optimization
コスト最適化

Operational
Excellence
運用上の
優秀性

Sustainability
持続可能性

マルチリージョンの活用

別リージョンにデータベースのレプリカを配置するなどして、データアクセスを分散する。



6つ目の柱
サステイナビリティ

サステナビリティの追加

2021年12月に初の変更点として6つ目の柱であるサステナビリティを追加

Reliability
信頼性

Performance
Efficiency
パフォーマンス
効率

Security
安全性

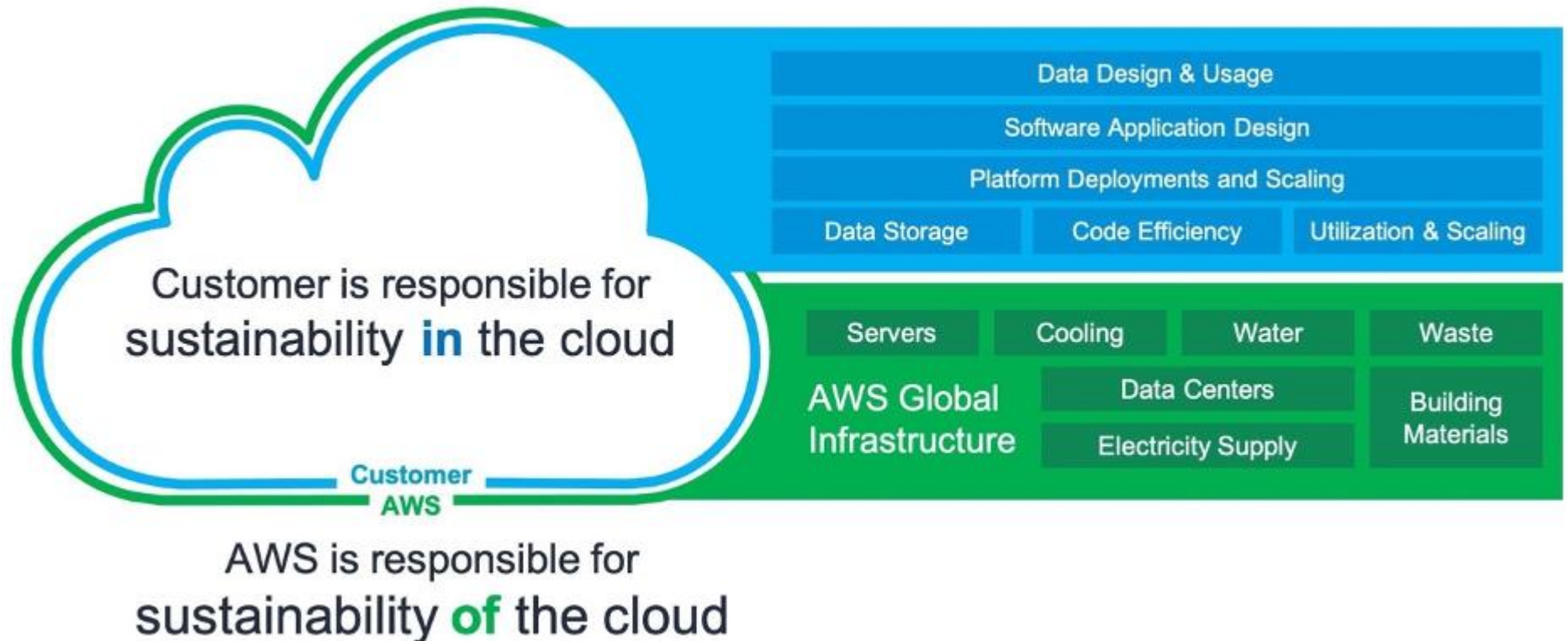
Cost
Optimization
コスト最適化

Operational
Excellence
運用上の
優秀性

Sustainability
持続可能性

サステナビリティの責任共有モデル

サステナビリティの追及はAWS責任共有モデルにより、ユーザーとAWS側の連携によって推進られる。



サステナビリティの目標

AWSはクラウドのサステナビリティの推進によって、インフラに関連するエネルギー使用率を80%削減する。

サーバー使用率

電力と冷却の効率

カスタム
データセンター
の設計

2025年までに100%
再生可能エネルギー
でAWSを運用

Customer is responsible for
sustainability **in** the cloud

Customer
AWS

AWS is responsible for
sustainability **of** the cloud

Servers
AWS Global
Infrastructure

Cooling
Data Centers
Electricity Supply

Water
Waste
Building
Materials

AWS側の主な対応

サステナビリティの目標

ユーザー側はクラウドを利用した効率的なアプリケーション設計やサービス選択を実施する。

ユーザー側の主な対応

効率的な
ストレージ
技術の使用

コード効率化

効率的な
インフラのデプロイ
とスケーリング

効率的な
アプリケーション
デザイン

データの
効率的な
デザイン
活用

サステナビリティの設計原則

ユーザーは次の設計原則を意識してサステナビリティを推進

影響を把握する

ビジネス上の成果と関連するサステナビリティへの影響を測定し、パフォーマンス指標を確立し、改善を評価し、提案された変更が与える影響を推定する。

サステナビリティ目標の設定

ワークロードごとに長期的な目標を設定し、投資収益率 (ROI) をモデル化し、サステナビリティ目標に投資するリソースを所有者に提供する。ユーザーごと、オペレーションごとなど、作業当たりの影響を軽減するために、拡大を計画し、アーキテクチャを設計する。

使用率の最大化

各ワークロードのサイズを適切に調整して、基盤となるハードウェアのエネルギー効率を最大化し、アイドル状態のリソースを最小限に抑える。

より効率的な新しいハードウェアとソフトウェアの提供を予測して採用する

パートナーによるアップストリームの改善をサポートし、ハードウェアとソフトウェアの選択の効率性を継続的に評価し、新しいテクノロジーを長期にわたって採用できる柔軟性を考慮して設計する。

マネージドサービスの使用

共有サービスは、幅広いワークロードをサポートするために必要なインフラストラクチャの量を削減。マネージドサービスを活用してアクセス頻度の低いデータをコールドストレージに移動したり、性能を調整したりするなどして、影響を最小限に抑え、サステナビリティのベストプラクティスを自動化する。

クラウドワークロードのダウンストリームの影響を軽減

サービスの使用に必要なエネルギーやリソースの量を削減し、ユーザーがデバイスをアップグレードする必要性を減らす。また、デバイスファームを使用したテストで影響を測定し、テストして、それらへの実際の影響を把握する。

サステイナビリティのベストプラクティス

ユーザーは次のベストプラクティスに従ってAWSを利用することでサステイナビリティを推進

- ユーザーの所在地に合わせて、ワークロードの地理的配置を最適化
- 時間やリソースを最も多く消費するコード領域を最適化
- ユーザーのデバイスや機器への影響を最適化
- データ分類ポリシーを実装
- ライフサイクルポリシーを使用して不要なデータを削除
- ネットワーク間でのデータ移動を最小化
- GPU の使用を最適化
- 潜在的なサステナビリティの改善を迅速に導入できる開発方法とテスト方法を採用
- ビルド環境の使用率を向上

アソシエイト試験への影響

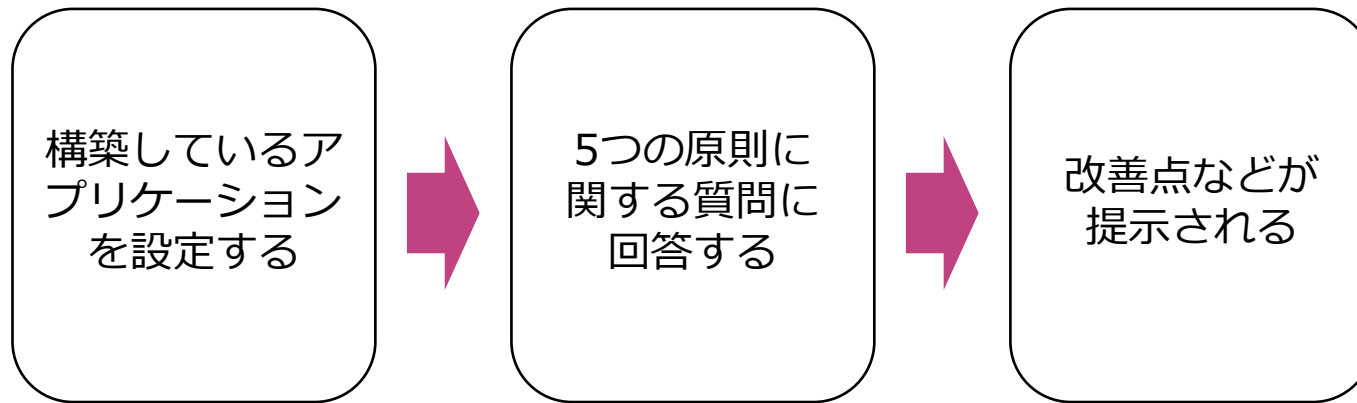
即座に問題に反映されているわけではないが、公式に試験範囲がアップデートされた場合に反映される可能性がある。

AWSが更新した場合にこの講座範囲をアップデートします。

Well-Architected Toolの確認

Well-Architected Toolの利用方法

Well-Architected Toolは構築中または運用中のアプリケーションがベストプラクティスに沿っているかを検証できるツール



Well-Architected Toolの定義

Well-Architected Toolの独特の用語を理解して、利用していく

ワークロード	ウェブサイト、ECサイト、モバイルアプリのバックエンド、分析プラットフォームなどビジネス価値をもたらす一連のコンポーネントの集合のこと。
マイルストーン	設計、テスト、実稼働、本稼働の製品ライフサイクルを通して進歩するアーキテクチャでの重要な変更内容のこと
レンズ	ベストプラクティスに照らしてアーキテクチャを評価し、改善すべき分野を特定する一貫した方法
高リスクの問題 (HRI)	ビジネスに重大な悪影響を及ぼす可能性があるアーキテクチャおよび運用上の問題
中リスクの問題 (MRI)	その程度は HRI より低いビジネス上の問題